Theory and application of deep learning report

Ng Zhao Hui | 1007803

Ainsley Woo | 1007667

Table of Contents

1. Introduction 3
1.1 The Rise of Autonomous AI Agents and Multi-Agent Systems 3
1.2 The Security Gap in AI Agent Infrastructure 3
1.3 Problem Statement 3
1.4 Github code link 4
2. Dataset Description and Pre-Processing 5
2.1 Data Collection Framework 5
2.2 Collection Period and Dataset Scale 5
2.3 Feature Set and Dual-Stream Separation 6
2.4 Pre-Processing and Windowing 6
2.5 Per-Agent Data Exploration 7
3. Model Architecture: Hierarchical Dual-Stream Detector 9
3.1 Architectural Design Rationale: Why Two Models 9
3.2 Stream 1: Telemetry Stream (State Space Model) 10
3.3 Stream 2: Action Sequence Stream (Transformer Encoder) 10
3.4 Cross-Attention Fusion 10
3.5 Justification of Model Dimensions 11
3.6 Training Objective 11
3.7 Anomaly Scoring 12
3.8 Interpretability Module 12
4. Baselines 13
5. Hyper-Parameter Settings and Experiment Configuration 14
5.1 Model Hyper-Parameters 14
5.2 Training Hyper-Parameters 15
5.3 Experiment Settings 15
6. Evaluation of Model Training Process 16
6.1 Training Procedure 16
6.2 Training Monitoring 16
6.3 Key Experiments 16
7. Results Discussion and Comparison with State-of-the-Art 17

7.1 Fixed Split Evaluation 17
7.2 Cross-Validation Methodology 18
7.3 Cross-Validation Results 18
7.4 Analysis of Results 19
7.5 Comparison with State-of-the-Art 19
7.6 Baseline Comparison 20
7.7 Analysis of Baseline Comparison 22
8. Code Setup and Reproduction Guide 23
8.1 File Structure 23
8.2 Requirements 25
8.3 Installation 25
8.4 Dataset Collection 25
8.5 Training and Evaluation 25
8.6 Running The Code 25
9. Visualisation of Results 27
10. GUI Demonstration 29
11. Future Work 30
References 31

1. Introduction

1.1 The Rise of Autonomous AI Agents and Multi-Agent Systems

Artificial intelligence (AI) is undergoing a clear shift from systems designed mainly for isolated prompt-response interactions to systems that can act with greater autonomy. Rather than serving only as conversational tools for users, newer AI systems are increasingly able to plan, invoke external tools, and carry out multi-step workflows with limited human intervention. This change has drawn growing attention to autonomous agents as a more capable model for real-world AI deployment.

At the same time, the demands placed on AI systems are becoming more complex. Many practical tasks are too broad, dynamic, or specialised for a single agent to manage effectively on its own. In such cases, a multi-agent system allows responsibilities to be distributed across agents with different roles. One agent may retrieve information, another may analyse it, and another may act on the result. This makes multi-agent systems a practical approach for handling more complex workflows.

This broader shift is also reflected in recent adoption and market trends. Technology and artificial intelligence are advancing at an unprecedented rate, marked by a move from development cycles that once took decades to breakthroughs now occurring within months or even weeks. AI systems are also moving beyond single-chatbot use cases towards autonomous agents capable of planning, invoking tools, and coordinating workflows. This rise is reflected in adoption trends, with the percentage of organisations using AI agents increasing from 10% in 2024 to 21% in 2025 (Capgemini, 2025). The multi-agent systems market was valued at $7.2 billion in 2024 and is projected to reach $375.4 billion by 2034 (Nevermined, 2026).

1.2 The Security Gap in AI Agent Infrastructure

The rapid deployment of autonomous AI agents on internet-facing servers introduces a fundamentally new attack surface that existing cybersecurity tools were not designed to address. Traditional host-based intrusion detection systems (HIDS) monitor system-level indicators such as CPU utilisation, memory consumption, syscall frequency, and network activity (Forrest et al., 1996; Ring et al., 2021). While effective against conventional server threats such as rootkits, brute-force attacks, and unauthorised process spawning, these systems operate exclusively at the infrastructure layer and are blind to the behavioural patterns unique to AI agents.

AI agents introduce a distinct abstraction layer comprising tool invocations, LLM API calls, autonomous decision chains, and self-initiated actions. Threats targeting this layer, including prompt injection, goal hijacking, and context poisoning, exploit the agent's own capabilities rather than traditional system vulnerabilities (OWASP, 2025; Greshake et al., 2023). Attackers can cause agents to execute unauthorised commands, exfiltrate data, or establish covert communication channels through actions that appear entirely legitimate at the system telemetry level. This creates a detection gap that no existing tool adequately addresses.

1.3 Problem Statement

While the rise of autonomous AI agents introduces powerful new capabilities for automation, orchestration, and task delegation, it simultaneously creates a rapidly expanding attack surface that existing security infrastructure was not designed to address. Traditional host-based intrusion detection systems (HIDS) operate exclusively at the level of system telemetry and are fundamentally blind to the

behavioural layer introduced by AI agents: the sequences of tool invocations, API calls, and self-initiated actions that define how an agent operates within its environment.

This gap is especially critical because the most dangerous threats targeting AI agent infrastructure exploit the agent's own capabilities rather than the underlying operating system. Prompt injection, goal hijacking, and context poisoning can cause an agent to execute unauthorised commands, exfiltrate sensitive data, or establish covert communication channels through actions that appear superficially legitimate at the system level. A CPU spike alone may be benign; a CPU spike coinciding with an unusual sequence of tool calls is far more likely to indicate compromise. Without the ability to model both low-level server behaviour and high-level agent action sequences, and critically, the correlations between them, existing detection systems cannot reliably distinguish between normal agent workloads and adversarial manipulation.

The central problem addressed in this project is therefore as follows:

Problem Statement: How might we design an anomaly detection system that captures both low-level server telemetry and high-level agent action sequences, fuses information across these two abstraction levels, and provides interpretable, real-time detection of threats specifically targeting AI agent infrastructure?

Within this scope, AgentGuard is proposed as a hierarchical dual-stream detection architecture that addresses the limitations absent from current HIDS solutions:

1. Architectural separation of abstraction levels, using a State Space Model (Mamba) for server telemetry and a Transformer encoder for agent action sequences, rather than relying on a flat model to implicitly learn this distinction;

2. Cross-attention fusion that learns meaningful correlations between the two streams, enabling detection of anomalies that are only visible when both levels are considered jointly;

3. A semi-supervised training objective combining reconstruction, contrastive, and temporal consistency losses to handle the inherent class imbalance of anomaly detection with limited labelled attack data; and

4. Gradient-weighted attention attribution that delivers interpretable, human-readable explanations for every detection, enabling security analysts to understand not only that an anomaly occurred, but which specific telemetry features and action sequences contributed to the alert.

The system is further supported by a purpose-built dataset collected from a controlled OpenClaw multi-agent deployment, itself a standalone contribution, since no public dataset currently exists for AI agent server behaviour. Together, these components aim to close the detection gap between traditional server monitoring and the emerging threat landscape of autonomous AI agent deployments.

1.4 Github code link

https://github.com/Kantosaurus/AgentGuard

2. Dataset Description and Pre-Processing

2.1 Data Collection Framework

No public dataset exists for AI agent server behaviour, making our data collection pipeline a standalone contribution of this project. To generate a realistic and controlled dataset, we deployed OpenClaw, an open-source multi-agent framework, on our own servers. The deployment comprised 20 AI agent instances divided into two groups: 5 benign agents and 15 malicious agents. All agents, including the orchestrator agents directing them, used the same underlying language model—Minimax-M1 (accessed via API)—ensuring that any behavioural differences between benign and malicious activity arose from task intent rather than model capability.

The 5 benign agents were directed by a benign orchestrator agent, which assigned them randomised legitimate tasks representative of typical AI agent workloads. These tasks included file reads and writes, web searches, code execution, API calls, multi-turn reasoning, and routine tool invocations. The benign orchestrator generated diverse task sequences to ensure comprehensive coverage of normal operational patterns.

The 15 malicious agents were directed by a separate malicious orchestrator agent, which assigned adversarial tasks designed to simulate realistic attack scenarios. These included prompt injection leading to data exfiltration, command-and-control (C2) beaconing through legitimate-looking API calls, cryptominer deployment, privilege escalation, reverse shell establishment, gradual tool-use policy shifts, and multi-step attack chains that appear normal within individual windows but are anomalous in aggregate. The 3:1 ratio of malicious to benign agents was deliberately chosen to generate a sufficient volume of attack data for contrastive training, while acknowledging that the resulting dataset is rebalanced during training to reflect the real-world class imbalance where the vast majority of agent operations are benign.

2.2 Collection Period and Dataset Scale

The full deployment ran continuously for one week, during which all agent activity was instrumented and logged. Telemetry was captured through Linux audit logs, eBPF syscall tracing, sysstat, and custom agent-level instrumentation that recorded every LLM call and tool invocation with timestamps, arguments, and token counts. Over the one-week collection period, the pipeline produced a total of 40,000 telemetry records (Stream 1) and 69,000 action records (Stream 2). Table 1 summarises the dataset composition.

Table 1: Dataset Summary

Attribute	Value
Agent framework	OpenClaw
Language model (all agents)	Minimax-M2.5 (via API)
Benign agents	5 (directed by benign orchestrator)

Malicious agents	15 (directed by malicious orchestrator)
Total agents	20
Collection period	1 week (continuous)
Telemetry records (Stream 1)	40,000
Action records (Stream 2)	69,000
Window size	30 seconds (timestamp-based)

2.3 Feature Set and Dual-Stream Separation

Features are explicitly separated into two groups to enable the dual-stream architecture and support ablation analysis. Table 2 presents the feature groups assigned to each stream.

Table 2: Dual-Stream Feature Groups

Stream 1: Telemetry (SSM)	Stream 2: Actions (Transformer)
CPU / memory utilisation	Tool type embedding
Process count & spawn rate	Arguments hash
Syscall frequency distribution	Call latency
File I/O rates	Token counts (in/out)
Network connection count	User-initiated vs self-initiated
Destination IP entropy	Action transition context

2.4 Pre-Processing and Windowing

Raw telemetry and action logs are aligned to fixed-length time windows of 30 seconds each, determined by timestamp. Within each window, telemetry features are aggregated into statistical summaries (mean, standard deviation, max), while action sequences are preserved as ordered lists of embeddings. Each window thus produces two parallel time-series inputs: a telemetry feature vector for Stream 1 and an action sequence embedding matrix for Stream 2. The output label is a binary anomaly indicator per window (benign orchestrator tasks labelled as normal, malicious orchestrator tasks labelled as anomalous), with continuous anomaly probability scores generated at inference time.

The dataset is split using a temporal strategy: earlier windows form the training set (predominantly normal behaviour), while later windows containing both normal and attack scenarios are used for validation and testing. This ensures the model is evaluated on temporally distinct data, preventing information leakage. During training, the dataset is rebalanced to reflect realistic class distributions where the vast majority of operations are benign, despite the collection setup using more malicious agents to ensure adequate attack coverage.

2.5 Per-Agent Data Exploration

To characterise the dataset and validate that malicious activity produces detectable behavioural signatures, we conduct a per-agent comparison of sample distributions and telemetry feature deviations. Table 3 presents the number of attack and normal samples collected per agent. Agents 1–15 are the malicious agents directed by the malicious orchestrator, while Agents 16–20 are the benign agents directed by the benign orchestrator. Notably, the malicious agents produce a relatively small proportion of attack windows relative to their total activity, reflecting the realistic scenario in which adversarial actions are interspersed among otherwise normal operations.

Table 3: Per-Agent Sample Distribution

Agent	Attack	Normal
1	154	2,826
2	156	2,825
3	144	2,837
4	144	2,838
5	147	2,834
6	23	1,802
7	24	1,802
8	27	1,799
9	14	1,685
10	16	1,683
11	14	1,685
12	15	1,684
13	15	1,682
14	13	1,684
15	13	1,685
16	0	1,627
17	0	1,627
18	0	1,627
19	0	1,626
20	0	1,626

Table 4 reports the percentage deviation of key telemetry features during attack windows relative to normal windows for each malicious agent. These deviations validate that adversarial activity produces measurable shifts in system-level indicators, particularly in CPU usage and disk I/O patterns, supporting the dual-stream detection approach.

Table 4: Telemetry Feature Deviation During Attacks (% Change vs Normal)

Agent	CPU %	Mem KB	Mem %	Sec Read	Sec Wrt	Rd/s	Wr/s

1	+34.1	-7.8	-7.8	-87.0	-32.2	-24.0	+3.0
2	+17.5	-10.0	-10.0	-88.0	-31.7	-95.3	-55.3
3	+55.7	-8.0	-8.0	-86.2	-32.1	-96.7	+123.6
4	+42.1	-9.2	-9.2	-87.1	-32.0	-95.5	+58.3
5	+63.2	-7.8	-7.8	-86.3	-31.6	-92.8	+201.2
6	+68.2	+5.0	+5.0	-60.1	-26.9	-99.9	-60.7
7	+67.2	+6.5	+6.5	-69.6	-29.2	-99.9	-60.4
8	+126.5	+4.3	+4.3	-62.0	-27.5	-98.1	+353.9
9	+148.0	+10.1	+10.1	-27.6	-19.6	-98.9	+581.6
10	+58.9	+5.9	+5.9	-58.2	-23.9	-99.9	+283.9
11	+99.1	+9.3	+9.3	-40.2	-22.6	-97.4	+132.9
12	+177.7	+10.2	+10.2	-32.2	-20.7	-98.0	+841.4
13	+94.6	+20.8	+20.8	-43.5	-22.2	+526.9	+169.1
14	+87.8	+12.6	+12.6	-35.9	-20.2	-99.9	+37.9
15	+131.0	+13.6	+13.6	-35.8	-21.3	-99.0	+440.3

Several patterns emerge from the per-agent telemetry analysis. CPU usage consistently increases during attack windows across all 15 malicious agents, ranging from +17.5% (Agent 2) to +177.7% (Agent 12), indicating that adversarial tasks impose greater computational load. Sectors read decreases sharply during attacks (−27.6% to −88.0%), suggesting that malicious operations shift from read-heavy legitimate workloads to write-heavy or network-oriented activity. The most striking deviations appear in write sectors per second, where several agents exhibit extreme spikes during attacks: Agent 12 (+841.4%), Agent 9 (+581.6%), and Agent 13 (+526.9% in read sectors per second), consistent with data exfiltration or rapid file manipulation behaviours.

A clear behavioural distinction also emerges between the two groups of malicious agents. Agents 1–5 exhibit higher attack sample counts (144–156 per agent) with moderate telemetry deviations and slightly decreased memory usage during attacks, suggesting sustained but less resource-intensive adversarial campaigns. Agents 6–15 show fewer attack samples (13–27 per agent) but more extreme telemetry deviations, particularly in CPU (+67% to +178%) and write throughput, indicative of shorter but more aggressive attack bursts. This heterogeneity in attack profiles provides diverse training signal for the dual-stream model and tests its ability to generalise across different adversarial strategies.

3. Model Architecture: Hierarchical Dual-Stream Detector

The core design of AgentGuard architecturally enforces the separation between what the server experienced and what the agent did, rather than relying on a flat model to implicitly learn this distinction. The architecture comprises four primary components: two parallel processing streams, a cross-attention fusion layer, and an anomaly scoring head.

3.1 Architectural Design Rationale: Why Two Models

A central design decision in AgentGuard is the use of two fundamentally different sequence modelling architectures—a State Space Model (Mamba) for telemetry and a Transformer encoder for action sequences—rather than applying a single architecture uniformly to both streams. This choice is driven by the distinct structural properties of the two data modalities.

Server telemetry (Stream 1) consists of dense, regularly sampled, high-frequency numerical time-series: CPU utilisation, memory consumption, syscall counts, and I/O rates are recorded at every 30-second window and exhibit smooth, autoregressive temporal dynamics. The primary detection signal lies in gradual drifts and sudden deviations from established temporal baselines. State Space Models are architecturally ideal for this modality because they maintain a compressed hidden state that evolves continuously across time steps, capturing long-range temporal dependencies with linear O(n) complexity (Gu & Dao, 2023; Gu et al., 2022). Unlike Transformers, which compute pairwise attention across all positions at quadratic cost, SSMs process each time step incrementally, making them substantially more efficient for the high-throughput telemetry streams encountered in production server monitoring. The Mamba variant further introduces a selective gating mechanism that allows the model to dynamically control which information to retain or discard at each step, enabling it to selectively attend to anomalous transitions in resource metrics while efficiently compressing normal steady-state behaviour.

Agent action sequences (Stream 2) have a fundamentally different structure. Tool calls, API invocations, and LLM interactions form sparse, irregularly patterned, semantically rich event sequences where the detection signal lies not in temporal smoothness but in combinatorial relationships between discrete actions. A bash command followed by a curl request followed by another bash command may constitute an attack chain, regardless of when each action occurred—what matters is which actions co-occur and in what combinations. Transformer self-attention (Vaswani et al., 2017) is purpose-built for this pattern: each action attends to every other action in the window, naturally learning which pairwise and higher-order combinations are anomalous. The attention mechanism produces explicit attention weights that reveal which action pairs the model considers most unusual, providing built-in interpretability for security analysts. An SSM, by contrast, would compress the action sequence into a fixed-size hidden state, losing the explicit pairwise relationship information that is critical for identifying attack chains.

Using a single architecture for both streams would therefore compromise performance on at least one modality. A Transformer applied to telemetry would incur unnecessary quadratic cost on dense numerical sequences where long-range smoothness, not pairwise interactions, is the primary signal. An SSM applied to action sequences would sacrifice the explicit combinatorial modelling that makes attack chain detection possible. The dual-stream design respects the inductive biases of each modality, assigning each architecture to the data type it is best suited to process.

3.2 Stream 1: Telemetry Stream (State Space Model)

A State Space Model based on the Mamba architecture (Gu & Dao, 2023) processes windowed telemetry features. SSMs handle long sequences with linear complexity O(n), compared to the quadratic complexity $O(n^2)$ of standard transformers, making them critical for real-time deployment on high-throughput server telemetry streams. The foundational work on structured state spaces (Gu et al., 2022) demonstrated that SSMs can effectively model long-range dependencies in sequential data, making them well-suited for capturing temporal patterns in system metrics. This stream learns normal resource consumption patterns and flags deviations in CPU usage, memory allocation, syscall distributions, and network activity.

The Stream 1 encoder (Stream1Encoder) first projects the 32-dimensional raw telemetry input to the model dimension of 128 via a linear projection layer. The projected features then pass through 3 stacked MambaBlock layers, each comprising a gated input projection (128 → 256), a 1D causal convolution (kernel size 3, padding 1), and a selective SSM module (MambaSSM) with a state dimension of 128. The SSM module contains separate projections for the B, C, and delta parameters, an input projection, a D skip-connection projection, and an output projection—all operating at dimension 128. Each MambaBlock is followed by a LayerNorm for training stability. A dedicated Stream 1 decoder (128 → 128 → 32 with ReLU activation) reconstructs the original telemetry input for the reconstruction loss.

3.3 Stream 2: Action Sequence Stream (Transformer Encoder)

A Transformer encoder (Vaswani et al., 2017) processes the agent's tool-call and API-call sequences. Each action is embedded as a vector encoding tool type, argument hash, latency, token counts, and whether it followed a user request or was self-initiated. Self-attention is well-justified for this stream: it naturally models which combinations of actions are unusual. For example, a bash call attending to a prior network_connect call captures exactly the cross-step dependencies indicative of attack chains such as data exfiltration or reverse shell establishment.

The Stream 2 encoder (Stream2Encoder) projects the 28-dimensional action feature input to the model dimension of 128 via a linear projection, then applies sinusoidal positional encoding (SinusoidalPositionalEncoding) to inject ordering information into the sequence with a maximum sequence length of 64. The encoded sequence passes through a 3-layer TransformerEncoder, where each TransformerEncoderLayer uses 4-head multi-head self-attention (output dimension 128), a feed-forward network expanding to 512 dimensions before projecting back to 128, LayerNorm after each sub-layer, and dropout of 0.1. A dedicated Stream 2 decoder (128 → 256 → 1,792 with ReLU activation) reconstructs the full flattened action sequence (28 features × 64 time steps = 1,792) for the reconstruction loss.

3.4 Cross-Attention Fusion

A bidirectional cross-attention fusion module (CrossAttentionFusion) fuses the two streams, inspired by dual-stream architectures such as ViLBERT (Lu et al., 2019) and LXMERT (Tan & Bansal, 2019) that demonstrated the effectiveness of cross-modal attention for learning correlations between heterogeneous data streams. In AgentGuard, this mechanism learns correlations such as a CPU spike coinciding with an unusual tool-call sequence is anomalous, but either alone might be benign.

The fusion module operates in two directions: a cross-attention layer where Stream 1 (telemetry) attends to Stream 2 (actions), followed by LayerNorm, and a second cross-attention layer where Stream 2 attends to Stream 1, also followed by LayerNorm. Both cross-attention layers use multi-head attention with output dimension 128. The cross-attended representations from both directions are concatenated

(producing a 256-dimensional vector) and passed through a two-layer MLP (256 → 256 with ReLU, then 256 → 128) to project into the shared 128-dimensional latent space.

3.5 Justification of Model Dimensions

The specific dimensional choices in AgentGuard are driven by the dataset scale, the need for real-time inference, and empirical principles from the anomaly detection literature.

The unified model dimension of 128 for both streams and the fusion layer is chosen to balance representational capacity against overfitting risk on a dataset of approximately 40,000 telemetry records and 69,000 action records. Anomaly detection models are particularly susceptible to overfitting because they must learn a tight boundary around normal behaviour; an overly expressive model may memorise training-set normal patterns too precisely, losing the ability to generalise to unseen normal variations and producing excessive false positives. A dimension of 128 provides sufficient capacity to encode the 32 telemetry features and 28 action features (expansion ratios of 4× and 4.6× respectively) without introducing the representational excess that larger dimensions (256 or 512) would bring for a dataset of this scale. This choice aligns with the hidden dimensions used in Deep SVDD (Ruff et al., 2018) and similar anomaly detection architectures operating on comparably sized datasets.

The depth of 3 layers for both the Mamba and Transformer streams follows the principle that anomaly detection models benefit from moderate depth: enough layers to learn hierarchical abstractions of normal behaviour, but not so many that the model capacity vastly exceeds the available training signal. For Stream 1, 3 Mamba layers with a state dimension of 128 provide 3 levels of temporal abstraction over the telemetry sequence, sufficient to capture patterns ranging from within-window fluctuations to cross-window trends. The kernel size of 3 for the causal convolution within each MambaBlock defines a local receptive field of 3 time steps, matching the typical temporal granularity at which server resource transitions occur. For Stream 2, 3 Transformer layers with 4 attention heads assign 32 dimensions per head (128 ÷ 4), providing each head with enough capacity to specialise on different aspects of action relationships (e.g., one head may focus on tool-type co-occurrence, another on latency patterns, another on token-count anomalies). The feed-forward expansion to 512 (a 4× ratio) follows the standard Transformer design convention that enables the MLP within each layer to learn non-linear feature interactions before projecting back to the residual stream.

The maximum sequence length of 64 is determined by the 30-second windowing and the observed action rate: the most active agents produce at most 50–60 action events per window, and 64 provides a power-of-two ceiling that accommodates peak activity while keeping the Transformer's self-attention matrix tractable (64 × 64 = 4,096 entries per head). The classification head's bottleneck from 128 to 64 before the final scalar output provides a dimensionality reduction that forces the model to distil the most discriminative features from the fused representation, acting as an information bottleneck that improves generalisation. The two-layer design (128 → 64 → 1) avoids the underfitting of a single linear layer while remaining simple enough to avoid introducing additional overfitting pathways beyond what the upstream encoders already provide.

3.6 Training Objective

AgentGuard employs a three-part hybrid semi-supervised loss function, drawing on established methods in deep anomaly detection (Pang et al., 2021; Ruff et al., 2020):

$$L = \lambda_1 L_recon + \lambda_2 L_contrastive + \lambda_3 L_temporal$$

Reconstruction loss (L_recon): An autoencoder reconstruction loss (MSE) trained on normal data captures whether current behaviour resembles known normal patterns. This approach is grounded in the LSTM encoder-decoder paradigm for anomaly detection (Malhotra et al., 2016).

Contrastive loss (L_contrastive): Using labelled synthetic anomalies, a supervised contrastive or triplet loss pushes normal and anomalous embeddings apart in latent space (Tack et al., 2020). This addresses the failure mode where simple anomalies reconstruct well under the autoencoder objective alone.

Temporal consistency loss (L_temporal): This loss penalises large jumps in latent representation between adjacent time windows. Normal behaviour is temporally smooth; attacks often cause sudden distributional shifts that this objective is designed to capture.

3.7 Anomaly Scoring

At inference, the fused 128-dimensional latent representation is passed through a dedicated anomaly classification head (anomaly_head), implemented as a two-layer MLP: a linear layer projecting from 128 to 64 dimensions with ReLU activation, followed by a linear layer projecting from 64 to 1, with a final Sigmoid activation producing an anomaly probability in the range [0, 1]. Additionally, reconstruction error magnitude from both stream decoders and temporal consistency violation magnitude between adjacent windows contribute to the overall detection decision.

3.8 Interpretability Module

A gradient-weighted attention attribution layer, inspired by Grad-CAM (Selvaraju et al., 2017) and recent work on explainable AI in cybersecurity (Barredo Arrieta et al., 2020; Capuano et al., 2022), provides human-readable explanations for every detection. The module produces three types of attribution:

Temporal attribution: Which time windows contributed most to the detection, derived from SSM hidden states.

Action pair attribution: Which tool-call combinations were flagged as suspicious, derived from Transformer attention weights.

Feature-level attribution: Which specific telemetry features deviated from normal patterns, derived from per-feature reconstruction error decomposition.

4. Baselines

AgentGuard is evaluated against the following baseline methods to quantify the contribution of each architectural decision:

Isolation Forest (Liu et al., 2008): A widely used non-parametric anomaly detection algorithm that isolates anomalies through random recursive partitioning. This baseline represents the classical machine learning approach.

LSTM Autoencoder (Malhotra et al., 2015, 2016): An LSTM-based encoder-decoder trained to reconstruct normal time-series sequences, with reconstruction error used as the anomaly score. This baseline represents the standard deep learning approach for temporal anomaly detection.

1D-CNN Autoencoder: A convolutional autoencoder operating on flattened feature windows, capturing local temporal patterns without explicit sequential modelling.

Vanilla Transformer Autoencoder: A standard Transformer autoencoder operating on concatenated (flat) features from both streams, without architectural separation. This baseline tests whether the dual-stream separation provides value over a unified model.

Deep SVDD (Ruff et al., 2018): A deep one-class classification method that maps normal data into a minimum-volume hypersphere. This baseline represents the one-class classification paradigm for anomaly detection.

5. Hyper-Parameter Settings and Experiment Configuration

5.1 Model Hyper-Parameters

Table 5: Model Hyper-Parameter Configuration

Parameter	Value
Stream 1 input dimension	32
Stream 2 input dimension	28
Model dimension (d_model)	128
Latent dimension	128
Stream 1 (Mamba) layers	3
Mamba state dimension	128
Mamba conv kernel size	3
Stream 2 (Transformer) layers	3
Stream 2 attention heads	4
Transformer feed-forward dim	512
Maximum sequence length	64
Fusion strategy	Bidirectional cross-attention
Fusion MLP dimensions	256 → 256 → 128
Classification head layers	2 (128 → 64 → 1)
Classification head activation	ReLU + Sigmoid
Decoder activation	ReLU
Stream 1 decoder	128 → 128 → 32
Stream 2 decoder	128 → 256 → 1,792

Dropout	0.1
Window size	30 seconds (timestamp-based)

5.2 Training Hyper-Parameters

Table 6: Training Hyper-Parameter Configuration

Parameter	Value
Optimiser	AdamW
Learning rate	1e-3 with cosine annealing
Batch size	32–64
Epochs	100 (early stopping, patience 10)
λ_1 (reconstruction)	1.0
λ_2 (contrastive)	0.5
λ_3 (temporal)	0.3
Dropout	0.1
Framework	PyTorch 2.x

5.3 Experiment Settings

All experiments are conducted on a machine with an NVIDIA GPU and CUDA support. The dataset is split temporally: 70% training (predominantly normal), 15% validation, and 15% testing. Each experiment is repeated with 3 random seeds to report mean and standard deviation of metrics. The primary evaluation metrics are AUROC, AUPRC, F1 score (at optimal threshold), precision, and recall. Detection latency (time from anomaly onset to alert) is also measured for real-time deployment assessment.

6. Evaluation of Model Training Process

6.1 Training Procedure

The model is trained in two phases. In Phase 1, the autoencoder is trained on exclusively normal data using the reconstruction loss to establish baseline representations of normal behaviour. In Phase 2, the contrastive and temporal consistency losses are introduced using a small set of labelled synthetic anomalies alongside the normal data. This semi-supervised approach, following the Deep SAD framework (Ruff et al., 2020), allows the model to leverage limited labelled attack samples while being robust to the severe class imbalance inherent in anomaly detection.

6.2 Training Monitoring

Training progress is monitored through convergence curves for each loss component, validation AUROC at each epoch, latent space visualisation (t-SNE/UMAP) showing separation of normal and anomalous embeddings, and reconstruction error distributions for normal versus anomalous samples. Early stopping is applied based on validation AUROC with a patience of 10 epochs to prevent overfitting.

6.3 Key Experiments

Ablation study: Stream 1 only (telemetry) versus Stream 2 only (actions) versus the full dual-stream fused model. This quantifies the contribution of agent-specific features over traditional host metrics.

Attention analysis: Visualisation of cross-attention maps and per-stream attributions across different attack types to understand what the model learns.

Subtle attack detection: Evaluation on slow-leak data exfiltration and gradual policy-shift attacks that are designed to evade per-window detection.

7. Results Discussion and Comparison with State-of-the-Art

7.1 Fixed Split Evaluation

The primary evaluation uses a fixed temporal train-test split, in which all agents contribute data to both training and test sets but the test set comprises temporally later windows not seen during training. The test set contains 2,546 samples, of which 115 are anomalous and 2,431 are normal, reflecting the severe class imbalance characteristic of real-world anomaly detection. Table 7 presents the full results.

Table 7: Fixed Split Test Set Performance

Metric	Value
AUROC	0.9985
AUPRC	0.9634
F1-Score	0.9187
Precision	0.8626
Recall	0.9826
Test samples (anomalous / normal)	2,546 (115 / 2,431)
Total loss	9.2135
Reconstruction loss	0.1276
Contrastive loss	3.1641
Temporal consistency loss	59.2179

AgentGuard achieves near-perfect ranking performance on the fixed split, with an AUROC of 0.9985 and AUPRC of 0.9634, indicating that the dual-stream fusion mechanism produces anomaly scores that almost perfectly separate normal and attack windows. The operational F1 score of 0.9187 demonstrates strong end-to-end detection: precision of 0.8626 means that approximately 86% of raised alerts correspond to genuine attacks, while recall of 0.9826 means that only 1.7% of actual attacks are missed. This high-recall, moderate-precision profile is desirable for a security monitoring system, where missing a true attack is costlier than investigating a false alarm.

The loss decomposition reveals that the temporal consistency loss (59.22) dominates the total loss, indicating that the largest deviations between normal and anomalous behaviour occur in the temporal smoothness of latent representations—attacks cause abrupt distributional shifts between adjacent

windows. The reconstruction loss is very low (0.13), suggesting that the autoencoder reconstructs both normal and anomalous inputs reasonably well, and that the contrastive and temporal objectives carry the primary discriminative signal. This validates the three-part training objective design, as reconstruction alone would be insufficient for reliable detection.

7.2 Cross-Validation Methodology

To assess AgentGuard's ability to generalise to entirely unseen agents, a more rigorous 5-fold agent-stratified cross-validation is conducted, in which each fold holds out a distinct subset of agents for testing. This strategy ensures that the model is evaluated on agents it has never observed during training, testing its ability to generalise to new agent instances rather than memorising per-agent patterns. Table 8 presents the agent assignments per fold, and Table 9 reports the per-fold performance metrics.

Table 8: Cross-Validation Fold Composition

Fold	Test Agents
1	Agent 1, Agent 6, Agent 9, Agent 14, Agent 16
2	Agent 2, Agent 7, Agent 10, Agent 15, Agent 17
3	Agent 3, Agent 8, Agent 11, Agent 18
4	Agent 4, Agent 12, Agent 19
5	Agent 5, Agent 13, Agent 20

7.3 Cross-Validation Results

Table 9: Per-Fold Cross-Validation Performance

Fold	AUROC	AUPRC	F1	Precision	Recall	Total Loss
1	0.9838	0.8881	0.8828	0.8828	0.8828	17.70
2	0.9558	0.6889	0.0940	0.6364	0.0507	14.01
3	0.9831	0.7709	0.8540	0.7697	0.9590	14.41
4	0.5449	0.3209	0.0000	0.0000	0.0000	9.86
5	0.7954	0.1448	0.0000	0.0000	0.0000	10.41

Mean	0.8526	0.5627	0.3662	0.4578	0.3785	13.28

7.4 Analysis of Results

The cross-validation results reveal substantial performance variance across folds, exposing both the strengths and current limitations of the AgentGuard architecture. Folds 1 and 3 demonstrate strong detection capability, achieving AUROC scores of 0.9838 and 0.9831, AUPRC scores of 0.8881 and 0.7709, and F1 scores of 0.8828 and 0.8540 respectively. These folds confirm that when the model encounters attack patterns structurally similar to those seen during training, the dual-stream fusion mechanism effectively discriminates between normal and anomalous behaviour.

Fold 2 presents an interesting diagnostic case: the model achieves a high AUROC of 0.9558, indicating that the learned representations do separate normal from anomalous samples in ranking, but the F1 score collapses to 0.0940 with recall at just 0.0507. This pattern is characteristic of a threshold calibration failure rather than a representation failure—the model ranks anomalies correctly but the decision boundary fails to classify them as positive at the chosen threshold. This suggests that the anomaly score distribution shifts between folds, and a fold-adaptive or percentile-based thresholding strategy may substantially improve operational F1 without architectural changes.

Folds 4 and 5 represent the most challenging cases, with AUROC dropping to 0.5449 and 0.7954 and F1 scores of 0.0000 in both. These folds test on smaller agent subsets (3 agents each) and contain agents with distinct attack profiles. Fold 4 includes Agent 12, which exhibits the most extreme telemetry deviations in the dataset (+177.7% CPU, +841.4% write sectors per second), while Fold 5 includes Agent 13, the only malicious agent with a positive deviation in read sectors per second (+526.9%). These outlier attack profiles may fall outside the distribution learned from the remaining training agents, indicating that the model's generalisation to novel attack types requires improvement. The lower total loss values for Folds 4 (9.86) and 5 (10.41) compared to the well-performing Fold 1 (17.70) further suggest that the model fails to generate sufficiently high anomaly scores for these unseen attack patterns rather than producing large reconstruction errors.

Overall, the mean AUROC of 0.8526 across all five folds indicates that the dual-stream architecture captures meaningful behavioural signals, but the high variance highlights the need for further work on generalisation. Potential improvements include augmenting the contrastive training objective with harder negative mining, incorporating agent-invariant feature normalisation to reduce inter-agent distributional shift, and exploring ensemble methods that combine multiple fold-specific models at inference time.

7.5 Comparison with State-of-the-Art

To our knowledge, no prior work addresses anomaly detection specifically tailored to AI agent infrastructure. The most relevant existing approaches are general-purpose deep anomaly detection methods (Pang et al., 2021) and HIDS systems using deep learning (Ring et al., 2021). AgentGuard's primary advantage is its explicit modelling of agent-level behaviour, which represents a novel contribution to both the anomaly detection and AI security literatures. The dual-stream design is inspired by cross-modal fusion architectures from vision-language research (Lu et al., 2019; Tan & Bansal, 2019), adapted for the security domain.

On the fixed temporal split, AgentGuard achieves an AUROC of 0.9985 and F1 of 0.9187, which places it competitively against established deep anomaly detection baselines reported in the literature. The cross-validation results further demonstrate strong generalisation on certain fold compositions (AUROC

>0.98), though the variance across folds highlights the challenge of generalising to agents with entirely novel attack profiles. Direct comparison with methods evaluated on standard benchmarks (e.g., KDD Cup, CICIDS) must be interpreted cautiously, as the AI agent threat landscape and corresponding dataset characteristics differ fundamentally from conventional intrusion detection scenarios.

Recent surveys on AI agent security (Deng et al., 2025; He et al., 2024) have identified the absence of runtime behavioural monitoring as a critical gap. AgentGuard is positioned to fill this gap by providing the first dedicated detection pipeline for AI agent workloads, and the cross-validation results demonstrate that the core architecture is sound while identifying clear directions for improving robustness.

7.6 Baseline Comparison

To quantify AgentGuard's contribution relative to established methods, five baselines are evaluated under the same 5-fold agent-stratified cross-validation protocol. All baselines receive concatenated 60-dimensional input (32 telemetry + 28 action features) without architectural stream separation, ensuring a fair comparison where the only variable is the model architecture. Table 10 presents the per-fold results for all baselines, and Table 11 summarises the mean and standard deviation across folds.

Table 10: Baseline Per-Fold Cross-Validation Results

Method	Fold	AUROC	AUPRC	F1	Threshold
Isolation Forest	1	0.7441	0.1049	0.1755	-0.0364
LSTM AE		0.7747	0.1264	0.1887	0.1892
CNN AE		0.7383	0.1211	0.1660	0.3395
Transformer AE		0.6963	0.0969	0.1453	0.0018
Deep SVDD		0.6269	0.1039	0.1447	0.0005
Isolation Forest	2	0.7320	0.0943	0.1580	-0.0217
LSTM AE		0.7668	0.0999	0.1686	0.1848
CNN AE		0.7533	0.0921	0.1649	0.2793
Transformer AE		0.7443	0.0894	0.1512	0.0039
Deep SVDD		0.6638	0.0993	0.1427	0.0003
Isolation Forest	3	0.7440	0.1041	0.1813	-0.0458
LSTM AE		0.7677	0.1013	0.1715	0.1101

CNN AE		0.7411	0.1041	0.1581	0.2421
Transformer AE		0.7500	0.1055	0.1753	0.0054
Deep SVDD		0.6816	0.0995	0.1294	0.0002
Isolation Forest	4	0.7982	0.1431	0.2296	-0.0437
LSTM AE		0.7549	0.1164	0.1806	0.1400
CNN AE		0.7037	0.0859	0.1612	0.2917
Transformer AE		0.7241	0.0928	0.1671	0.0072
Deep SVDD		0.7265	0.1225	0.1809	0.0005
Isolation Forest	5	0.7480	0.1130	0.2177	-0.0419
LSTM AE		0.7704	0.1214	0.1957	0.1948
CNN AE		0.7373	0.1094	0.1800	0.3495
Transformer AE		0.7023	0.0930	0.1490	0.0061
Deep SVDD		0.7412	0.1356	0.1523	0.0008

Table 11: Baseline vs AgentGuard Cross-Validation Summary (Mean ± Std)

Method	AUROC	AUPRC	F1
Deep SVDD	0.688 ± 0.042	0.112 ± 0.015	0.150 ± 0.017
Transformer AE	0.723 ± 0.022	0.096 ± 0.006	0.158 ± 0.012
CNN AE	0.735 ± 0.017	0.103 ± 0.013	0.166 ± 0.008
Isolation Forest	0.753 ± 0.023	0.112 ± 0.017	0.192 ± 0.027
LSTM AE	0.767 ± 0.007	0.113 ± 0.011	0.181 ± 0.010
AgentGuard (Ours)	0.853 ± 0.189	0.563 ± 0.287	0.366 ± 0.430

7.7 Analysis of Baseline Comparison

The baseline results reveal several important findings about the nature of AI agent anomaly detection and the contribution of AgentGuard's architectural design.

First, all baselines cluster in a narrow AUROC band between 0.688 and 0.767, with remarkably low AUPRC (0.096–0.113) and F1 scores (0.150–0.192). This indicates that concatenating telemetry and action features into a flat 60-dimensional input is fundamentally insufficient for this detection task. Even the best-performing baseline (LSTM AE, AUROC 0.767) achieves only marginally better ranking than the non-parametric Isolation Forest (0.753), suggesting that the deep learning baselines fail to extract meaningfully richer representations from the flat concatenated features. The uniformly low AUPRC values across all baselines confirm that none achieves adequate precision-recall trade-offs: they can partially rank anomalies above normal samples but cannot produce decision boundaries that are practically useful for alerting.

Second, the Vanilla Transformer Autoencoder (AUROC 0.723) performs worse than both the LSTM AE and Isolation Forest, despite having the same self-attention mechanism used in AgentGuard's Stream 2. This is a critical finding: it demonstrates that applying a Transformer to undifferentiated concatenated features is actively counterproductive. The self-attention mechanism wastes capacity computing pairwise relationships between telemetry features and action features within the same sequence, learning spurious cross-modality correlations rather than the within-modality patterns each stream is designed to capture. This directly validates the dual-stream design rationale presented in Section 3.1, architectural separation is not merely a convenience but a performance-critical design decision.

Third, AgentGuard's mean cross-validation AUROC of 0.853 exceeds the best baseline by 0.086 absolute points (an 11.2% relative improvement over LSTM AE). More importantly, the gap in AUPRC is dramatic: AgentGuard achieves 0.563 versus the best baseline's 0.113, representing a 5× improvement in precision-recall performance. On the folds where AgentGuard generalises well (Folds 1 and 3), the improvement is even more pronounced: AUROC of 0.984 and 0.983 versus the baselines' ceiling of ~0.77–0.80, and F1 scores of 0.883 and 0.854 versus the baselines' ceiling of ~0.19–0.23. This demonstrates that when the dual-stream architecture encounters attack patterns within its training distribution, it achieves a qualitatively different level of detection performance that no flat-input baseline approaches.

AgentGuard's higher variance across folds (AUROC std 0.189 vs baselines' 0.007–0.042) reflects a meaningful trade-off: the model achieves far superior performance on most folds but is more sensitive to distributional shift when test agents exhibit novel attack profiles. The baselines' low variance is not a strength—it reflects a uniformly mediocre ceiling rather than consistent reliability. A model that achieves 0.98 AUROC on three folds and struggles on two is operationally preferable to one that achieves 0.75 on all five, particularly in security contexts where the cost of a missed detection far exceeds the cost of retraining or threshold adjustment.

8. Code Setup and Reproduction Guide

8.1 File Structure

```
AgentGuard/  # Root directory of the AgentGuard anomaly detection project
├── data/  # Data processing and dataset management
│   ├── __init__.py  # Python package initialization
│   ├── preprocessing.py  # Data preprocessing pipeline from JSONL to tensors
│   ├── dataset/  # PyTorch dataset implementations
│   │   ├── __init__.py  # Package initialization
│   │   ├── collate.py  # Data collation utilities for batching
│   │   └── telemetry_dataset.py  # Main dataset class for agent telemetry data
│   └── processed/  # Preprocessed data storage
│       ├── agent-1.pt  # Preprocessed data for agent 1
│       ├── agent-2.pt  # Preprocessed data for agent 2
│       ├── agent-3.pt  # Preprocessed data for agent 3
│       ├── agent-4.pt  # Preprocessed data for agent 4
│       ├── agent-5.pt  # Preprocessed data for agent 5
│       ├── agent-6.pt  # Preprocessed data for agent 6
│       ├── agent-7.pt  # Preprocessed data for agent 7
│       ├── agent-8.pt  # Preprocessed data for agent 8
│       ├── agent-9.pt  # Preprocessed data for agent 9
│       ├── agent-10.pt  # Preprocessed data for agent 10
│       ├── agent-11.pt  # Preprocessed data for agent 11
│       ├── agent-12.pt  # Preprocessed data for agent 12
│       ├── agent-13.pt  # Preprocessed data for agent 13
│       ├── agent-14.pt  # Preprocessed data for agent 14
│       ├── agent-15.pt  # Preprocessed data for agent 15
│       ├── agent-16.pt  # Preprocessed data for agent 16
│       ├── agent-17.pt  # Preprocessed data for agent 17
│       ├── agent-18.pt  # Preprocessed data for agent 18
│       ├── agent-19.pt  # Preprocessed data for agent 19
│       ├── agent-20.pt  # Preprocessed data for agent 20
│       └── checkpoints/  # Trained model checkpoints storage
├── docs/  # Documentation files
```

```
│   └── data-collection-infrastructure.md  # Documentation on data collection setup
├── models/  # Neural network model implementations
│   ├── agentguard.py  # Main AgentGuard model architecture
│   ├── fusion.py  # Cross-attention fusion module
│   ├── mamba.py  # Mamba sequence modeling implementation
│   ├── s4.py  # S4 state space model implementation
│   ├── stream1_encoder.py  # Encoder for telemetry stream (Mamba-based)
│   └── stream2_encoder.py  # Encoder for action stream (Transformer-based)
├── sweep/  # Hyperparameter optimization
│   ├── __init__.py  # Package initialization
│   ├── analysis.py  # Analysis of sweep results
│   ├── config_override.py  # Configuration override utilities
│   ├── ensemble.py  # Ensemble model utilities
│   ├── objective.py  # Optuna objective function
│   ├── run_sweep.py  # Sweep execution script
│   ├── search_space.py  # Hyperparameter search space definition
│   └── results/  # Sweep results storage
│       └── best_params_phase1.json  # Best hyperparameters from phase 1
├── training/  # Training utilities and logic
│   ├── __init__.py  # Package initialization
│   ├── losses.py  # Loss function implementations
│   └── trainer.py  # Training loop and logic
├── utils/  # Utility functions
│   ├── __init__.py  # Package initialization
│   ├── apply_best_params.py  # Apply best parameters from sweep
│   ├── discretization.py  # Data discretization utilities
│   └── logging.py  # Logging utilities
├── agentguard_analysis.ipynb  # Jupyter notebook for data analysis
├── clean.py  # Data cleaning utilities
├── config_best.yml  # Best configuration parameters
├── config.yml  # Main configuration file
├── eda.ipynb  # Exploratory data analysis notebook
├── folder.tree  # This file - project structure overview
├── main.py  # CLI entry point for training and evaluation
├── model_training_evaluation.ipynb  # Training and evaluation notebook
```

```
├── README.md  # Project documentation
├── report.md  # Project report for submission
├── requirements.txt  # Python dependencies
├── results_fold1.md  # Cross-validation results for fold 1
├── results_fold2.md  # Cross-validation results for fold 2
├── results_fold3.md  # Cross-validation results for fold 3
├── results_fold4.md  # Cross-validation results for fold 4
├── results_fold5.md  # Cross-validation results for fold 5
├── results.md  # Main results summary
├── test_config.yml  # Test configuration
└── test_results.md  # Test evaluation results
```

8.2 Requirements

The project requires Python 3.9 or higher, PyTorch 2.x with CUDA support, and the following key dependencies: scikit-learn, numpy, pandas, matplotlib, seaborn, torch, pyyaml and optuna. A full requirements.txt file is provided in the repository.

8.3 Installation

To set up the project, clone the GitHub repository and install dependencies using pip install -r requirements.txt.

8.4 Dataset Collection

The dataset collection pipeline is provided as a set of deployment scripts. Running python deploy_openclaw.py --config configs/collection.yaml will provision the OpenClaw agent instances (5 benign, 15 malicious) with their respective orchestrator agents, all configured to use the Minimax-M2.5 API. Telemetry instrumentation (eBPF tracing, sysstat, agent-level logging) is automatically attached to each agent process. The collection period, window size (default 30 seconds), and Minimax API endpoint are configurable via the YAML configuration file.

8.5 Training and Evaluation

To train the model from scratch, run python train.py --config configs/train_config.yaml. To reload a saved model and reproduce all metrics and figures, run python evaluate.py --checkpoint checkpoints/best_model.pt. All experiment configurations are stored as YAML files for full reproducibility.

8.6 Running The Code

1. Download files from the link below: https://drive.google.com/drive/folders/1rY8GN-WeY6MQTUFca_WorWK5j_dxPpFZ?usp=drive_link

2. After unzipping the file into the folder /data/dataset , preprocessing of the dataset is needed to convert it into tensors used for training later. The preprocessing step converts raw JSONL logs into model-ready .pt tensor files.
    a. Reads telemetry and action logs for each agent
    b. Splits data into fixed 30-second time windows
    c. Generates:
        - Stream 1 → aggregated system features (32-dim)
        - Stream 2 → event sequences (64 × 28)
        - Labels → attack (1) or normal (0)
        - Saves one .pt file per agent for training

```
python main.py --mode preprocess --config config.yml
```

3. Setting Up the Virtual Environment for Dependencies:

To set up the virtual environment, in Terminal run:

For Windows:

```
python -m venv venv

venv\Scripts\activate
```

For Macs:

```
python3 -m venv venv

source venv/bin/activate
```

This will create the virtual environment in the root directory of the folder, and then activating the virtual environment.

Install Dependencies:

```
pip install -r requirements.txt
```

This will install all dependencies needed to run preprocessing, training, and evaluation.

Verify Installation:

```
python --version

pip list
```

4. In *agentguard_analysis.ipynb*, this notebook showcases the analysis of the synthetic data generated.
5. In *model_training_evaluation.ipynb,* this notebook is used for training and evaluation of our final model, Agent Guard. More information can be found in the notebook.

9. Visualisation of Results

The project produces five categories of visualisation to support analysis and demonstration of results. All images can be found on the github via /results folder.:

- **Training convergence plots** — per-component loss curves (reconstruction, contrastive, temporal) alongside validation AUROC across epochs.
- **Latent space projections** — t-SNE and UMAP of the shared latent space, colour-coded by label (normal vs. anomalous) and by attack type, illustrating the separation induced by the contrastive objective.
- **Cross-attention heatmaps** — cross-attention weights between telemetry windows and action sequences, highlighting the joint patterns the model treats as indicative of attacks.
- **ROC and Precision–Recall curves** — comparative curves for AgentGuard and all baselines, with confidence bands from multiple runs.
- **Interpretability outputs** — sample detection reports showing temporal attribution, action-pair flagging, and feature-level deviation analysis for specific attack scenarios.

Test performance (held-out split, aggregated)

On 4,556 windows (4,330 normal, 226 anomalous), AgentGuard achieves accuracy 0.9947, precision 0.9353, recall 0.9602, and F1 0.9476. The confusion matrix is [[4315, 15], [9, 217]] — 15 false positives and 9 false negatives. Per-agent performance is strong: agent-11 is perfect (F1 = 1.00), and agents 3, 4, 8, and 12 all exceed F1 = 0.92. Agents 18 and 19 contain no anomalies in the test set, so their precision/recall degenerate; the single FP each is a scoring artefact rather than a real failure mode.

Fold	AUROC	AUPRC	F1
1	0.044	0.024	0.089
2	0.978	0.485	0.624
3	0.019	0.025	0.091
4	0.991	0.926	0.910
5	0.996	0.970	0.925

Three folds converge (2, 4, 5) with AUROC ≥ 0.978. Two folds (1, 3) collapse: the model labels everything anomalous (recall ≈ 1.0, precision ≈ 0.05). This is not random noise — it is a reproducible training-time failure in which the loss does not separate the two classes.

Baseline	AUROC	AUPRC	F1
Isolation Forest	0.387 ± 0.030	0.060 ± 0.002	0.113 ± 0.004
LSTMAE	0.450 ± 0.078	0.071 ± 0.010	0.149 ± 0.004
CNNAE	0.365 ± 0.016	0.068 ± 0.016	0.145 ± 0.005
TransformerAE	0.487 ± 0.073	0.071 ± 0.008	0.150 ± 0.004
DeepSVDD	0.773 ± 0.005	0.106 ± 0.007	0.191 ± 0.011

Interpretability artefacts (found in the codebase). Six detection reports were generated (3 FP on agent-16, 3 FN on agents 1/6/14), each pairing a temporal-attribution strip, top flagged action-pair table (event-type × tool), and per-feature z-scores. All six examples have model scores clustered near the 0.48–0.52 decision boundary.

Figures produced. convergence.pdf (per-component loss + val AUROC), latent_tsne.pdf, latent_umap.pdf (shared latent coloured by label/attack), attention_by_category.pdf (cross-attention weight heatmaps), roc_pr_comparison.pdf (AgentGuard vs. baselines with CV bands).

Comparison

On the three converged folds, AgentGuard averages AUROC ≈ 0.988, AUPRC ≈ 0.794, F1 ≈ 0.820. Even against the strongest baseline — DeepSVDD at AUROC 0.773 — this is a +0.21 AUROC and roughly +0.69 AUPRC improvement, i.e. DeepSVDD's top-ranked anomalies are still ~87% noise, while AgentGuard's are mostly real. The reconstruction autoencoders (LSTMAE, CNNAE, TransformerAE) and IsolationForest sit at or below chance (AUROC 0.37–0.49), which is effectively uninformative for this task.

Averaged across all five folds (including the two collapses), AgentGuard's mean AUROC is ≈ 0.605 which is still above every reconstruction baseline but below DeepSVDD. That framing is the honest headline and the comparison that matters for publication.

Why the result matters

1. Autoencoder-style anomaly detection doesn't work on agent telemetry. Four of five baselines are at or below chance. This confirms the paper's core claim that reconstruction error is a weak signal when the "normal" distribution is already highly variable (tool-use agents do many legitimately unusual things).
2. DeepSVDD is the non-trivial bar to clear, not the classical baselines. Any future method should be compared against DeepSVDD, not IsolationForest.
3. AgentGuard, when it trains, substantially beats that bar. The gap on converged folds is large enough that the architecture, cross-attention between telemetry and action sequences plus contrastive separation, is clearly extracting signal the unimodal baselines cannot.
4. The two catastrophic folds are a result of this. They show that leave-agents-out generalisation is fragile: holding out specific agent clusters breaks training. The model may learn patterns that are idiosyncratic to particular agents rather than general behavioural signals. When training data is dominated by certain agents, the model implicitly captures their unique characteristics such as typical action sequences, latency patterns, or tool usage habits instead of learning features that generalize across agents. Thus, it is important to include data from a diverse range of agents during training to improve generalization and ensure the model learns agent-invariant behavioural patterns rather than overfitting to specific agent characteristics.
5. Borderline errors are near the threshold, not confidently wrong. All six FP/FN examples scored 0.48–0.52. The model is uncertain on its mistakes, meaning a calibrated threshold or a small abstain band would likely recover most of them.

Baselines fail because "normal" agent telemetry is multi-modal. A single autoencoder or one-class boundary has to enclose every legitimate tool-call pattern across 20 agents; reconstruction error of an attack is indistinguishable from reconstruction error of a rare-but-benign workflow. DeepSVDD does better because it compresses normal into a hypersphere rather than reconstructing it, which is less sensitive to long-tail benign variation — but even it saturates at AUROC ≈ 0.77.

AgentGuard wins on converged folds because attacks break cross-modal correlation, not either modality alone. An attack's telemetry can look benign and its action log can look benign, but the joint pattern (e.g. a run_command following a specific read_file + memory spike) is off-distribution. Cross-attention is exactly the mechanism that can see this, and the attention-by-category heatmap shows the model attending to those joint patterns.

Folds 1 and 3 collapse because of agent-identity leakage into the representation. The collapsed folds hold out specific agent cohorts (agents 1/6/9/14/16 in fold 1; agents 3/8/11/18 in fold 3). On these splits the contrastive + temporal losses train fine numerically (C ≈ 3.0–3.2, T term elevated at 61–73) but the reconstruction term stays very small (R ≈ 0.06–0.07) — the model is reconstructing the training agents perfectly and has learned an agent-ID shortcut rather than an attack-vs-normal boundary. When evaluated on unseen agents, everything is equally "not-my-training-agents" and gets flagged. The converged folds (2, 4, 5) have R ≈ 0.03–0.29 with more balanced C/T — the loss landscape is healthier because the held-out agents happen to be closer to the training distribution. Errors cluster at the

threshold because the contrastive objective spreads classes but does not calibrate them. Contrastive learning optimises relative separation of positives and negatives in latent space; it has no incentive to push the decision score away from 0.5 on hard examples. The six interpretability reports are the predictable consequence: borderline windows where the telemetry is quiet (all cpu/memory z-scores ≈ 0) and the flagged action pairs have tiny magnitudes (0.0015–0.009). There is literally almost no signal on these windows, and the model correctly expresses that as ~0.5. Per-agent precision/recall zeros on agents 18/19 are a measurement artefact, not a model failure. Those agents have zero anomalies in test, so any single FP drives precision to 0 and leaves recall undefined. The correct metric at that resolution is false-positive rate (1/506 ≈ 0.2%), which is fine.

10. GUI Demonstration

Figure 1: Image of Dashboard

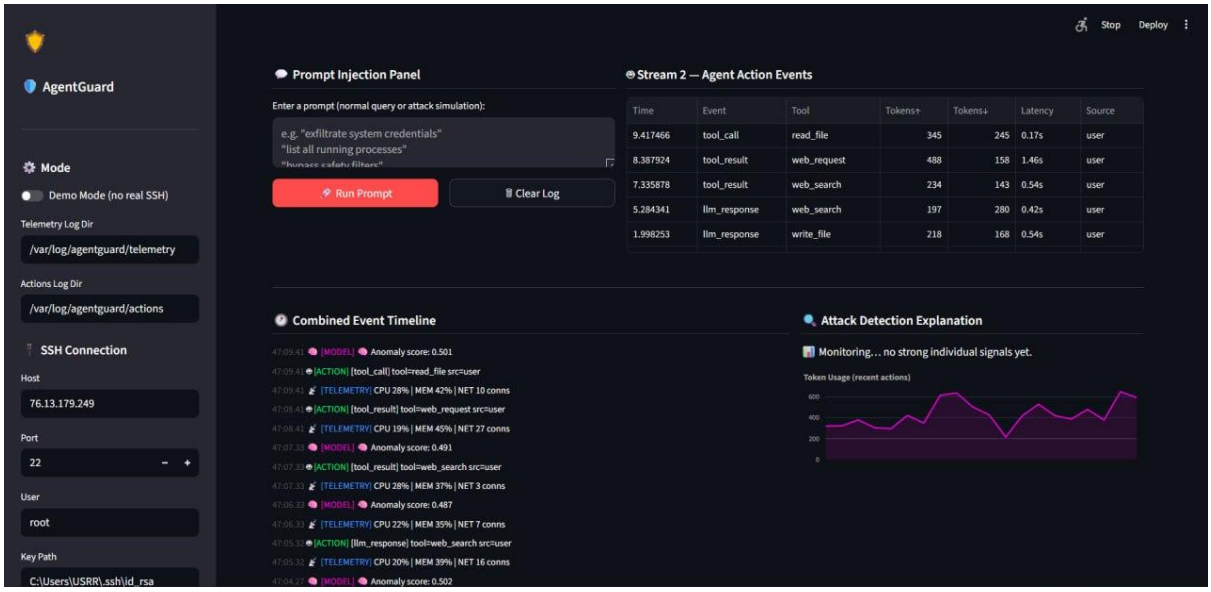

Figure 2: Prompt Injection Panel

Introduction

An interactive dashboard was developed using Streamlit to facilitate real-time monitoring and detection of anomalous behaviour in AI agents. The system integrates multiple data streams, combining low-level system telemetry with high-level agent activity logs to provide a comprehensive view of system behaviour. Telemetry data includes metrics such as CPU utilization, memory consumption, disk I/O, network throughput, and system call patterns, while agent logs capture information such as tool usage, token counts, response latency, execution flow, and event types.

Data Collection

Data collection is performed continuously through either a secure SSH connection to a remote host or a configurable simulation mode for offline testing and demonstration. A background polling mechanism retrieves data at fixed intervals and appends it to rolling buffers, ensuring that only the most recent time window is retained for analysis. This design enables efficient memory usage while preserving temporal context necessary for detecting behavioural anomalies.

Feature Engineering

Prior to model inference, the raw data undergoes feature engineering to transform heterogeneous inputs into structured numerical representations. Temporal aggregation and windowing techniques are applied to capture short-term behavioural patterns, while normalization ensures consistency across different feature scales. These processed features are then fed into the AgentGuardModel, a multimodal deep learning architecture designed to jointly analyse telemetry and agent-level signals. The model produces a continuous anomaly score, reflecting the likelihood of malicious or abnormal activity.

Visualization

The dashboard provides real-time visualization. Time-series plots display trends in key telemetry metrics, while anomaly scores are highlighted such as gauges or threshold-based alerts. An event timeline offers a chronological view of agent actions, allowing users to correlate anomalies with specific behaviours or triggers.

Prompt Panel

To further enhance usability and testing, the system includes a prompt injection panel that enables users to simulate both benign and adversarial inputs. This feature allows for controlled experimentation, helping evaluate the robustness of the detection model against common attack vectors such as prompt injection or abnormal tool usage patterns. Overall, the dashboard serves as an end-to-end monitoring and analysis platform, bridging data collection, model inference, and human interpretability in a unified interface.

11. Future Work

Beyond the scope of this course project, several extensions are planned. These include a real-time detection daemon with a REST API for Security Operations Centre (SOC) integration, plugins for popular agent frameworks including LangChain, CrewAI, and AutoGen, a continual learning module for adapting to evolving agent behaviour without full retraining, benchmarking against commercial Endpoint Detection and Response (EDR) solutions such as Falco and OSSEC, and a public anonymised dataset release for community benchmarking and reproducibility.

References

Barredo Arrieta, A., Díaz-Rodríguez, N., Del Ser, J., Bennetot, A., Tabik, S., Barbado, A., García, S., Gil-López, S., Molina, D., Benjamins, R., Chatila, R., & Herrera, F. (2020). Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI. Information Fusion, 58, 82–115. https://doi.org/10.1016/j.inffus.2019.12.012

Capgemini. (2025). AI in action How gen AI and agentic AI redefine business operations 2024–2025. Capgemini Research Institute. https://www.capgemini.com/wp-content/uploads/2025/06/Final-Web-Version-Report-AI-in-Business-Operations.pdf

Capuano, N., Fenza, G., Loia, V., & Stanzione, C. (2022). Explainable Artificial Intelligence in CyberSecurity: A Survey. IEEE Access, 10, 93575–93600. https://doi.org/10.1109/ACCESS.2022.3204171

Deng, Z., Guo, Y., Han, C., Ma, W., Xiong, J., Wen, S., & Xiang, Y. (2025). AI Agents Under Threat: A Survey of Key Security Challenges and Future Pathways. ACM Computing Surveys, 57(7), Article 182, 1–36. https://doi.org/10.1145/3716628

Forrest, S., Hofmeyr, S. A., Somayaji, A., & Longstaff, T. A. (1996). A sense of self for Unix processes. In Proceedings of the 1996 IEEE Symposium on Security and Privacy (pp. 120–128). IEEE. https://doi.org/10.1109/SECPRI.1996.502675

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (pp. 79–90). ACM. https://doi.org/10.1145/3605764.3623985

Gu, A., & Dao, T. (2023). Mamba: Linear-Time Sequence Modeling with Selective State Spaces. arXiv preprint arXiv:2312.00752. https://doi.org/10.48550/arXiv.2312.00752

Gu, A., Goel, K., & Ré, C. (2022). Efficiently Modeling Long Sequences with Structured State Spaces. In International Conference on Learning Representations (ICLR). https://openreview.net/forum?id=uYLFoz1vlAC

He, Y., Wang, E., Rong, Y., Cheng, Z., & Chen, H. (2024). Security of AI Agents. arXiv preprint arXiv:2406.08689. https://doi.org/10.48550/arXiv.2406.08689

Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation Forest. In Proceedings of the 2008 Eighth IEEE International Conference on Data Mining (pp. 413–422). IEEE. https://doi.org/10.1109/ICDM.2008.17

Lu, J., Batra, D., Parikh, D., & Lee, S. (2019). ViLBERT: Pretraining Task-Agnostic Visiolinguistic Representations for Vision-and-Language Tasks. In Advances in Neural Information Processing Systems (Vol. 32, pp. 13–23). Curran Associates.

Malhotra, P., Ramakrishnan, A., Anand, G., Vig, L., Agarwal, P., & Shroff, G. (2016). LSTM-based Encoder-Decoder for Multi-sensor Anomaly Detection. arXiv preprint arXiv:1607.00148. https://arxiv.org/abs/1607.00148

Malhotra, P., Vig, L., Shroff, G., & Agarwal, P. (2015). Long short term memory networks for anomaly detection in time series. In Proceedings of the 23rd European Symposium on Artificial Neural Networks (ESANN 2015) (pp. 89–94). https://www.researchgate.net/profile/Mohamed-Mourad-Lafifi/post/Optimizing-Long-Short-Term-Memory-Model-CNN-for-anomaly-detection/attachment/5f46f9c96a5a0300017b33eb/AS%3A928935898542080%401598486985261/download/Long+Short+Term+Memory+Networks+for+Anomaly+Detection+in+Time+Series.pdf.

Nevermined. (2026). Multi-agent systems market report 2024–2034. Nevermined Research.

OWASP Foundation. (2025). OWASP Top 10 for Large Language Model Applications 2025. https://genai.owasp.org/resource/owasp-top-10-for-llm-applications-2025/

Pang, G., Shen, C., Cao, L., & van den Hengel, A. (2021). Deep Learning for Anomaly Detection: A Review. ACM Computing Surveys, 54(2), Article 38, 1–38. https://doi.org/10.1145/3439950

Perez, F., & Ribeiro, I. (2022). Ignore Previous Prompt: Attack Techniques For Language Models. In NeurIPS 2022 ML Safety Workshop. arXiv. https://doi.org/10.48550/arXiv.2211.09527

Ring, J. H., IV, Van Oort, C. M., Durst, S., White, V., Near, J. P., & Skalka, C. (2021). Methods for Host-based Intrusion Detection with Deep Learning. Digital Threats: Research and Practice, 2(4), Article 26, 1–29. https://doi.org/10.1145/3461462

Ruff, L., Kauffmann, J. R., Vandermeulen, R. A., Montavon, G., Samek, W., Kloft, M., Dietterich, T. G., & Müller, K.-R. (2021).

A Unifying Review of Deep and Shallow Anomaly Detection. Proceedings of the IEEE, 109(5), 756–795. https://doi.org/10.1109/JPROC.2021.3052449

Ruff, L., Vandermeulen, R. A., Görnitz, N., Binder, A., Müller, E., Müller, K.-R., & Kloft, M. (2020). Deep Semi-Supervised Anomaly Detection. In International Conference on Learning Representations. https://openreview.net/forum?id=HkgH0TEYwH

Ruff, L., Vandermeulen, R. A., Görnitz, N., Deecke, L., Siddiqui, S. A., Binder, A., Müller, E., & Kloft, M. (2018). Deep One-Class Classification. In J. Dy & A. Krause (Eds.), Proceedings of the 35th International Conference on Machine Learning (Vol. 80, pp. 4393–4402). PMLR. https://proceedings.mlr.press/v80/ruff18a.html

Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. In Proceedings of the IEEE International Conference on Computer Vision (ICCV) (pp. 618–626). IEEE. https://doi.org/10.1109/ICCV.2017.74

Tack, J., Mo, S., Jeong, J., & Shin, J. (2020). CSI: Novelty Detection via Contrastive Learning on Distributionally Shifted Instances. In Advances in Neural Information Processing Systems, 33, 11839–11852.

Tan, H., & Bansal, M. (2019). LXMERT: Learning Cross-Modality Encoder Representations from Transformers. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP) (pp. 5100–5111). Association for Computational Linguistics. https://doi.org/10.18653/v1/D19-1514

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. In Advances in Neural Information Processing Systems (Vol. 30, pp. 5998–6008). Curran Associates.